

Codex

A portable, local-first scaffolding-and-sync tool for markdown documentation wikis.

Spec version	v1.1 (April 2026)
Tool version	v1.0 (initial build)
Language	Python 3.8+ (pure standard library)
External deps	None
Install path	D:\Claude Projects\Project Codex\
Reference impl.	Iron Soul / Planet Scoria Prime wiki v3.3
Document built	2026-05-01

Audience. This document is the consolidated reference for Codex — synthesized from CODEX_BUILD_SPEC_v1_1.md, codex-build-plan.html, INGEST_PROCEDURE.md, and SEMANTIC_LINT_PROCEDURE.md. It is intended for Claude Code and the human developer to share as a single readable artifact during the build.

Authority. If anything in this document contradicts the build spec, the build spec wins. Surface contradictions before acting on them.

Contents

1. What Codex Is
2. Architecture
 - 2.1 The three folder roles
 - 2.2 Wiki folder structure
 - 2.3 Frontmatter schema (the API)
 - 2.4 The 15 automation scripts
 - 2.5 Configuration files
3. Design Principles
4. The Three Operations
 - 4.1 Bootstrap
 - 4.2 Sync
 - 4.3 Ingest
5. File Manifest (What to Build)
6. Build Workflow (Phases 1–8)
7. Build Readiness Dashboard
 - 7.1 Summary statistics
 - 7.2 Build order (priority-sorted)
 - 7.3 Score-vs-priority gaps
8. Conflicts and Contradictions
9. Project Files Reference
10. Claude Code Behavior Rules
11. Procedures Reference (Ingest + Semantic Lint)
- A. Appendix A: Frontmatter field reference
- B. Appendix B: Quick command reference

1. What Codex Is

Codex is a **scaffolding and sync tool** for creating and maintaining markdown-based documentation wikis. One Codex installation, many consuming-project wikis. Each consuming project's wiki is its own folder of plain markdown files, openable in Obsidian or any markdown reader, with automated dashboards, validators, and integrity checks.

Codex itself is not a wiki. Codex is the tool that *creates* wikis, and that *cascades infrastructure improvements* to wikis it has created without touching their content.

What Codex produces

A two-tier documentation system:

- **Tier 1 — Developer Wiki (internal)** — browseable in Obsidian. Internal frameworks, technical references, contributing guides. Shareable with collaborators when ready.
- **Tier 2 — Confidential Profile** — single markdown file with project secrets, business strategy, and behavioral rules Claude follows when the file is loaded as project knowledge. Upload-ready.

Both tiers are plain markdown with YAML frontmatter, with a pure-Python automation layer providing 15 utility functions (dashboards, validators, integrity checks, canon consistency, sync, concept coverage).

Why Codex exists

Without Codex, a developer running multiple projects either (a) maintains documentation inconsistently across projects, or (b) reinvents the same documentation discipline repeatedly. Codex standardizes the discipline once and applies it everywhere.

The problems Codex solves:

- **Knowledge drift** — facts that change in one place but not in three others.
- **Confidentiality leaks** — internal terminology accidentally appearing in public docs.
- **Stale documentation** — a framework changed, but its briefing guide is still last quarter's version.
- **Loss of decision context** — choices made in chat that nobody remembers six months later.
- **Brain dump pollution** — speculative ideas accidentally treated as canonical.
- **Source drift** — new sources arriving but never integrated into the wiki (addressed by the v1.1 Ingest operation).
- **Multi-project documentation overhead** — every project needing the same structural setup from scratch.

2. Architecture

2.1 The three folder roles

```
D:/claude/ <-- workspace root
|
+-- _codex/ <-- THE TOOL. Exactly one of these.
|   +-- README.md
|   +-- CHANGELOG.md
|   +-- PROJECT_WIKI_BUILD_SPEC.md
|   +-- bootstrap.py Creates new wikis from templates
|   +-- _scripts/ Master copies of automation scripts
|   | +-- _lib/frontmatter.py Shared parser
|   +-- _config/ Generic config templates (commented)
|   +-- _template/ Page templates for starter content
|
+-- project-x/ <-- A consuming project
|   +-- (project x source code)/ Untouched by Codex
|   +-- wiki/ <-- THE WIKI for project x
|   +-- Home.md
|   +-- 00-Start-Here/
|   +-- 01-<Domain-1>/
|   +-- _scripts/ Synced from _codex/_scripts/
|   +-- _config/ Customized for project x
|   +-- _canon/ Project x ground-truth facts
|   +-- _sources/raw/ Project x ingested-source archive (v1.1)
|   +-- _confidential/ Project x secrets
|
+-- project-y/
|   +-- (project y source)/
|   +-- wiki/ <-- Same structure, different content
```

Category	Role	Count
_codex/	The tool itself	Exactly one
project-x/, project-y/, ...	Consuming projects	Many, one per thing you work on
project-x/wiki/, ...	Wikis (one per project)	One per project that wants docs

2.2 Wiki folder structure (what bootstrap.py creates)

```
<project>/wiki/
+-- README.md Top-level overview for humans
+-- Home.md Landing page; table of contents
+-- 00-Start-Here/ Overview, glossary, terminology, how-to
+-- 01-<Domain-1>/ Project-specific section (renamed in Phase 1)
+-- 02-<Domain-2>/ Another project-specific section
+-- 04-Contributing/ Update cascade, file routing, style guide, SPEC
+-- Attachments/ Images, diagrams, PDFs
+-- public/ Public-facing briefing-guide pairs
+-- scripts/ Project-specific scripts (e.g. exporters)
+-- _dashboards/ Auto-generated. Never hand-edit.
+-- _decisions/ Append-only decision log (inc. ingest-log.md)
+-- _inbox/ Ephemeral holding pen for sources awaiting triage
+-- _sources/raw/ Permanent, read-only archive of ingested sources
```

```
+-- _brain_dump/ Quarantined unvalidated ideas – NOT canon
+-- _canon/ Structured ground-truth facts (YAMLs)
+-- _context/ Claude operating rules for this wiki
+-- _scripts/ The 15 automation scripts (synced from Codex)
+-- _config/ YAML config for validators
+-- _confidential/ Personal + Claude profile. NEVER publish.
```

Folders prefixed `_` are infrastructure. When sharing a wiki externally, exclude every `_`-prefixed folder except optionally `_decisions`.

`_inbox/` vs `_sources/raw/` (v1.1). `_inbox/` is ephemeral — a triage area for sources that arrived but haven't been ingested yet. `_sources/raw/` is permanent and read-only — the archive of sources that *have* been ingested. `canon_sources` on wiki pages cite paths in `_sources/raw/`. Ingest moves files from `_inbox/` to `_sources/raw/` on success.

2.3 Frontmatter schema (the API)

Every content page has this YAML header. Scripts read frontmatter; they do not parse arbitrary page content for metadata.

```
---
title: "Human-Readable Page Title"
type: framework | reference | guide | overview | episode |
      dashboard | profile | brain_dump | decision | context_rules
visibility: internal | public | confidential
completion: 0-100
status: gap | outlined | solid | ready
phase: 0-N # optional
last_updated: YYYY-MM-DD
dependencies: ["Other-Page"]
public_pair: "public/Briefing.md" | null
blocking_questions: ["What is X?"]
canon_sources: ["_sources/raw/<file>.md section 2.1"] # required for status: ready
unverified_claims: [] # must be empty for status: ready
source: "Free-text citation (legacy)"
allow_forbidden_terms: true | false # optional escape hatch
generated: true # auto-gen dashboards only

# Brain dump entries only:
dump_status: exploring | validated | migrated | superseded | rejected
migrated_to: ["Path/To/Canon-Page.md"]
---
```

Status band rule: ready ≥ 80, solid 55–79, outlined 30–54, gap < 30.

Type durability convention (Principle #12). framework pages are load-bearing — they are the structural rules of a project, and consuming projects are expected to mirror every type: framework page to an external durable location (typically a Claude project knowledge file). reference, episode, and similar inventory types are derived from frameworks plus source material; their durability is only as good as the wiki itself. Operational types (guide, overview, dashboard, profile, brain_dump, decision, context_rules) are not part of the framework/reference split.

Source frontmatter (reduced, v1.1)

Files in `_inbox/` and `_sources/raw/` use a reduced schema:

```
---
source: "Original citation (URL, book, conversation, etc.)"
ingested_date: YYYY-MM-DD # date scaffolded, not date ingested
status: pending_triage | ingested
---
```

`scaffold_source.py` creates these with status: `pending_triage`. Ingest flips them to `ingested` on archive.

2.4 The 15 automation scripts

All pure Python stdlib. No pip install. Live in `_scripts/` of every wiki, with master copies in `_codex/_scripts/`.

#	Script	Purpose
1	<code>update_dashboards.py</code>	Master orchestrator — runs all aggregators and validators.
2	<code>build_completion_dashboard.py</code>	Red/yellow/green completion dashboard.
3	<code>validate_terminology.py</code>	Forbidden-terms scan, context-aware (all/audience/internal).
4	<code>validate_reveal_conceit.py</code>	Leak scan on visibility: public pages.
5	<code>validate_canon_integrity.py</code>	status: ready must have <code>canon_sources</code> , empty <code>unverified_claims</code> .
6	<code>check_cascade.py</code>	Source -> parallel doc mtime comparison.
7	<code>check_cross_refs.py</code>	Broken <code>[[links]]</code> + orphan pages.
8	<code>check_framework_briefing_sync.py</code>	Framework <-> public-pair staleness.
9	<code>check_canon_consistency.py</code>	Page contradictions vs <code>_canon/</code> AND page-to-page.
10	<code>collect_open_questions.py</code>	Roll up all <code>blocking_questions</code> across pages.
11	<code>steel_thread_tracker.py</code>	Multi-layer feature completion.
12	<code>build_canon_drift_report.py</code>	Snapshot + diff of <code>_canon/</code> over time.
13	<code>delta_source_docs.py</code>	Diff two source-doc versions + cascade impact.
14	<code>sync_from_kit.py</code>	Pull updated infrastructure from the Codex installation.
15	<code>check_concept_coverage.py</code> (v1.1)	Entities in <code>_canon/roster.yaml</code> mentioned in pages but lacking a dedicated page.
+	<code>scaffold_brain_dump.py</code>	Create a new brain dump entry.
+	<code>scaffold_source.py</code> (v1.1)	Create a new <code>_inbox/</code> entry with frontmatter for ingest.
+	<code>scaffold_</code> .py	Project-specific scaffolds (added per project).

2.5 Configuration files

`_config/` (behavior tuning, per-project)

- **`forbidden_terms.yaml`** — regex rules for trademark/naming violations, context-aware (all / audience / internal).
- **`reveal_leak_patterns.yaml`** — phrases that leak unreleased content. Severities: error / warning / info (info documents approved terms without flagging).
- **`cascade_map.yaml`** — source → derived doc propagation.
- **`steel_threads.yaml`** — multi-layer feature manifest.
- **`concept_coverage.yaml`** (v1.1, optional) — tunes `check_concept_coverage.py`. Keys: `min_mentions` (default 2), `exclude_folders`, `exclude_entities` (escape hatch).

`_canon/` (ground-truth facts, per-project)

- **`counts.yaml`** — every canonical number that appears more than once.
- **`roster.yaml`** — named entities with canonical names and aliases.
- **`taxonomy.yaml`** — structured classifications.
- **`timeline.yaml`** — milestones, versions, progression parameters.

The consistency checker reads `_canon/` as ground truth. Pages contradicting it are flagged. Pages contradicting *each other* (when `_canon/` doesn't arbitrate) are also flagged in a separate section.

3. Design Principles

1. **One idea per page.** If a page grows past ~400 lines, split it.
2. **Authoritative source $\neq$ wiki.** The wiki is derived. Source wins conflicts. Cite via `canon_sources`.
3. **Internal/public split is directional.** Internal $\rightarrow$ Public, never reverse. Strip internal terminology unless confirmed public-safe. Public pages link only to other public pages.
4. **Update cascade is directional.** Source $\rightarrow$ Derived $\rightarrow$ Wiki. Never reverse.
5. **Frontmatter is the API.** Scripts read frontmatter. Don't hardcode page metadata.
6. **Validators have escape hatches.** `allow_forbidden_terms: true` exists by design.
7. **Dashboards are expendable.** Regenerated every run.
8. **Canon is the floor, not the ceiling.** `_canon/` captures facts that **MUST** be consistent. Pages can contain more — they just can't contradict canon.
9. **Brain dump is quarantine.** `_brain_dump/` is explicitly NOT canon. Promotion is always explicit.
10. **Codex is the tool, not the content.** Codex never holds project content. It creates and updates wikis. Each wiki holds its own content.
11. **Ingest proposes, humans dispose (v1.1).** Canon writes require confirmation. Contradictions get flagged, not overwritten. Archived sources are read-only.
12. **Frameworks are load-bearing; references are derived inventory.** Pages with `type: framework` are the structural rules of a project — they should exist in both the wiki and an external durable location (e.g. project knowledge files uploaded to a Claude project). Pages with `type: reference` (and similar inventory types) are generated from frameworks plus sources, and are durable only to the extent that the wiki itself is. Losing a reference is recoverable from its framework; losing a framework is not.

4. The Three Operations Codex Performs

Codex performs exactly three operations against a consuming project. Bootstrap is one-time per project; Sync is occasional, initiated when Codex itself improves; Ingest is the most frequent operation on a live project, run whenever a new source enters orbit.

4.1 Bootstrap (one-time per project)

```
cd <wherever Codex is installed>
python bootstrap.py <target-wiki-path>
```

Creates the full wiki folder structure at <target-wiki-path>, copies all 15 scripts, drops in commented config templates, populates starter meta-pages (Home, Glossary, Terminology-Rules, How-to-Use-This-Wiki, Style-Guide, Update-Cascade, File-Routing, Confidential Profile skeleton, CLAUDE_CONTEXT_RULES, INGEST_PROCEDURE, SEMANTIC_LINT_PROCEDURE, and READMEs for canon/brain_dump/decisions/sources).

After bootstrap, the consuming project does Phases 3–7 of the build workflow (see section 6) to populate content.

4.2 Sync (ongoing, when Codex itself improves)

```
cd <wiki path>
python _scripts/sync_from_kit.py <path-to-Codex-installation>
```

Pulls updated infrastructure from Codex into the consuming wiki:

Operation	What it touches
Overwritten	_scripts/ (the 15 automation scripts)
Overwritten	04-Contributing/PROJECT_WIKI_BUILD_SPEC.md
Overwritten	_context/INGEST_PROCEDURE.md and _context/SEMANTIC_LINT_PROCEDURE.md (shipped procedures, not customized per project)
Merge-new-only	_config/ and _template/ files (existing customized files preserved; new template files added)
NEVER touched	Content folders (00-Start-Here/, 01-*/, _canon/, _sources/raw/, _confidential/, _decisions/, _brain_dump/, _dashboards/, _inbox/, public/, Home.md, README.md), _context/CLAUDE_CONTEXT_RULES.md (customized per project)

Sync refuses to run if the wiki has uncommitted git changes (override with `--force`). Sync supports `--dry-run` to preview changes.

4.3 Ingest (ongoing, once per new source) — v1.1

```
# In the consuming wiki:
python _scripts/scaffold_source.py <path-or-url>
# Then, in Claude Code:
# "Ingest _inbox/<source>.md"
```

Reads a new source document and integrates it into the wiki: extracts facts, routes them to `_canon/` (with confirmation) or to existing/new wiki pages, links related concepts, archives the original source, and appends an entry to `_decisions/ingest-log.md`.

The seven steps (full contract in `INGEST_PROCEDURE.md`):

- 1. Read.** Open the source in `_inbox/`. Confirm frontmatter (source, ingested_date, status: pending_triage).
- 2. Extract.** Walk the source and classify items: canon-worthy, page-worthy, mention-worthy, or noise. Present the classified plan to the user.
- 3. Route.** Canon-worthy → propose YAML; ASK before writing. Page-worthy → create new page with full frontmatter. Mention-worthy → update existing page. Contradictions → flag in ingest log; never overwrite silently.
- 4. Link.** Add `[[wikilinks]]` between related pages. Update `Home.md` only if a new top-level section was created.
- 5. Archive.** Move `_inbox/` → `_sources/raw/`. Set status: ingested, ingested_date: YYYY-MM-DD.
- 6. Log.** Append entry to `_decisions/ingest-log.md` naming date, source, pages created/updated, canon entries, contradictions.
- 7. Validate.** Run `python _scripts/update_dashboards.py` and confirm zero new validator errors.

Refuses to run if

- The source in `_inbox/` lacks required frontmatter. Run `scaffold_source.py` first.
- Canon contradictions are detected and the user has not confirmed either overwrite or quarantine.
- The wiki has uncommitted git changes from an unrelated session (override with `--force`).

Ingest supports `--dry-run` to preview the full routing plan without writing anything — recommended for the first ingest in any new project, and for any source over ~5,000 words.

5. File Manifest (What to Build)

The complete tree of files Codex must produce, organized by category. Template filenames in `_template/` use `|` as a path separator placeholder; `bootstrap.py` replaces `|` with `/` when copying templates into the destination wiki.

```
_codex/
+-- README.md Quickstart for humans
+-- CHANGELOG.md Version history (first line: "# v1.0 - <date>")
+-- PROJECT_WIKI_BUILD_SPEC.md The portable spec (this document, copied)
+-- bootstrap.py Scaffolders
|
+-- _scripts/
| +-- _lib/
| | +-- __init__.py
| | +-- frontmatter.py Shared frontmatter + YAML-subset parser
| |
| +-- update_dashboards.py Orchestrator
| +-- build_completion_dashboard.py
| +-- validate_terminology.py
| +-- validate_reveal_conceit.py
| +-- validate_canon_integrity.py
| +-- check_cascade.py
| +-- check_cross_refs.py
| +-- check_framework_briefing_sync.py
| +-- check_canon_consistency.py
| +-- collect_open_questions.py
| +-- steel_thread_tracker.py
| +-- build_canon_drift_report.py
| +-- delta_source_docs.py
| +-- sync_from_kit.py
| +-- check_concept_coverage.py (v1.1)
| +-- scaffold_brain_dump.py
| +-- scaffold_source.py (v1.1)
|
+-- _config/ Templates with commented examples
| +-- README.md
| +-- forbidden_terms.yaml
| +-- reveal_leak_patterns.yaml
| +-- cascade_map.yaml
| +-- steel_threads.yaml
| +-- concept_coverage.yaml (v1.1, optional)
|
+-- _template/ Page templates for starter content
    +-- Home.md
    +-- 00-Start-Here|Project-Overview.md
    +-- 00-Start-Here|How-to-Use-This-Wiki.md
    +-- 00-Start-Here|Glossary.md
    +-- 00-Start-Here|Terminology-Rules.md
    +-- 04-Contributing|Update-Cascade.md
    +-- 04-Contributing|File-Routing.md
    +-- 04-Contributing|Style-Guide.md
    +-- _canon|README.md
    +-- _canon|counts.yaml
    +-- _canon|roster.yaml
    +-- _canon|taxonomy.yaml
    +-- _canon|timeline.yaml
    +-- _brain_dump|README.md
    +-- _decisions|README.md
    +-- _decisions|ingest-log.md (v1.1)
```

```
+-- _inbox|README.md
+-- _sources|README.md (v1.1)
+-- _sources|raw|README.md (v1.1)
+-- _context|CLAUDE_CONTEXT_RULES.md
+-- _context|INGEST_PROCEDURE.md (v1.1, ship verbatim)
+-- _context|SEMANTIC_LINT_PROCEDURE.md (v1.1, ship verbatim)
+-- _confidential|Confidential_Profile.md
```

INGEST_PROCEDURE.md and **SEMANTIC_LINT_PROCEDURE.md** ship verbatim from the separate files accompanying the build spec — do not paraphrase, do not shorten, do not 'improve.' They are Claude-facing contracts and their exact wording matters.

6. Build Workflow (Phases 1–8)

Phase 1 — Intake (BEFORE writing any code)

Ask the user the five intake questions and confirm answers before proceeding: (1) Where should Codex be installed? (2) Are you providing the Iron Soul reference implementation? (3) Codex version to start at? (4) Should bootstrap also git-init the new wiki? (5) Any project-specific constraints (OS, Python version, network)?

Phase 2 — Scaffold Codex

Create the directory structure from section 5. Empty folders are fine at this stage.

Phase 3 — Build the scripts

For each of the 15 scripts, implement against the contract in section 2.4. Critical implementation details: pure stdlib only (no PyYAML — implement a minimal YAML-subset parser); WIKI_ROOT = parent.parent.parent from inside `_lib/frontmatter.py`; validators must strip fenced code blocks and inline code before pattern-matching; frontmatter list fields (dependencies, blocking_questions, canon_sources, unverified_claims, migrated_to) are parsed as YAML-subset lists.

Phase 4 — Build the templates

Create page templates in `_template/` with complete frontmatter (including `canon_sources: []` and `unverified_claims: []`), placeholder content with `<Project Name>` markers, and YAML configs that parse but ship empty (`rules: []`) with commented examples. `_context/INGEST_PROCEDURE.md` and `_context/SEMANTIC_LINT_PROCEDURE.md` ship verbatim. `_context/CLAUDE_CONTEXT_RULES.md` is customized per project but MUST include a Question-Answering Behavior section with the four named rules.

Phase 5 — Build bootstrap.py and sync_from_kit.py

Both must refuse to operate destructively without confirmation, support `--dry-run`, and print clear summaries. Bootstrap must create `_sources/raw/` and `_sources/` folders with READMEs. Sync must overwrite `_context/INGEST_PROCEDURE.md` and `_context/SEMANTIC_LINT_PROCEDURE.md` but leave `_context/CLAUDE_CONTEXT_RULES.md` untouched.

Phase 6 — Self-test

Bootstrap a test wiki to a temporary path, run `update_dashboards.py` on it, confirm: all aggregators write to `_dashboards/`; all validators run with 0 errors; the bootstrap is reproducible (idempotent given the same inputs); `_sources/raw/` exists and is empty with a README; `_context/INGEST_PROCEDURE.md` and `_context/SEMANTIC_LINT_PROCEDURE.md` exist verbatim.

Phase 7 — Document

Write `README.md` with a one-paragraph 'what Codex is', quickstart commands (bootstrap a new wiki, sync an existing wiki, ingest a source), and pointer to `PROJECT_WIKI_BUILD_SPEC.md`. Write `CHANGELOG.md` starting with `# v1.0 — <today's date>`.

Phase 8 — Confirm with user

Show the user the file manifest, the directory tree at the install path, and the output of a successful self-test bootstrap. Confirm everything works before considering the build complete.

The four required Question-Answering rules

CLAUDE_CONTEXT_RULES.md in every bootstrapped wiki must include a Question-Answering Behavior section containing at least these four rules:

- Consult wiki pages before raw sources when answering questions about the project.
- Cite specific pages using [[wikilinks]] in answers.
- Flag uncertainty explicitly; never confabulate when canon is silent.
- For cross-source synthesis, name every page being synthesized.

7. Build Readiness Dashboard

Component-by-component readiness scoring against the v1.1 build spec, with a build-order priority for each item. Bands match the spec's own status system: **gap** < 30, **outlined** 30–54, **solid** 55–79, **ready** ≥ 80. Priority numbers (P1 → P54) reflect dependency order — lower P# means build it earlier.

7.1 Summary statistics

Average readiness score	81 / 100
Ready (≥ 80)	37 items
Solid (55–79)	17 items
Outlined (30–54)	0 items
Gap (< 30)	0 items
Total components	54 items

Read this as: the average readiness across all 54 buildable artifacts is **81/100**, putting the project in the **ready** band overall. There are no **gap**-band items, meaning no component lacks enough design clarity to begin work — though a handful of items in the **solid** band warrant pause-and-clarify moments before code is written (see section 7.3).

Tier breakdown

Tier	Items	Avg score	Band
T1 · Foundation (P1–3)	3	92/100	ready
T2 · Scripts & scaffolds (P4–20)	17	76/100	solid
T3 · Configs & canon (P21–31)	11	81/100	ready
T4 · Templates & procedures (P32–49)	18	84/100	ready
T5 · Entry points & ops tests (P50–54)	5	77/100	solid

7.2 Build order (priority-sorted)

The complete priority-sorted build order. Build P1 first, P54 last. If a script's dependencies aren't built yet, stop and back up.

P#	Tier	Type	Component	Score	Band
P1	T1	PY	_lib/frontmatter.py (YAML-subset parser)	80	ready
P2	T1	PY	CHANGELOG.md	95	ready
P3	T1	MD	PROJECT_WIKI_BUILD_SPEC.md (copied)	100	ready
P4	T2	PY	collect_open_questions.py	90	ready
P5	T2	PY	build_completion_dashboard.py	85	ready
P6	T2	PY	validate_terminology.py	85	ready
P7	T2	PY	validate_canon_integrity.py	85	ready
P8	T2	PY	validate_reveal_conceit.py	80	ready
P9	T2	PY	check_cross_refs.py	85	ready
P10	T2	PY	check_cascade.py	75	solid
P11	T2	PY	check_framework_briefing_sync.py	75	solid
P12	T2	PY	check_canon_consistency.py	70	solid
P13	T2	PY	check_concept_coverage.py [NEW]	75	solid
P14	T2	PY	steel_thread_tracker.py	65	solid
P15	T2	PY	build_canon_drift_report.py	65	solid
P16	T2	PY	delta_source_docs.py	60	solid
P17	T2	PY	scaffold_brain_dump.py	80	ready
P18	T2	PY	scaffold_source.py [NEW]	70	solid
P19	T2	PY	update_dashboards.py (orchestrator)	80	ready
P20	T2	PY	sync_from_kit.py	70	solid
P21	T3	MD	_config/README.md	85	ready
P22	T3	YML	_config/forbidden_terms.yaml	85	ready
P23	T3	YML	_config/reveal_leak_patterns.yaml	85	ready
P24	T3	YML	_config/cascade_map.yaml	80	ready
P25	T3	YML	_config/steel_threads.yaml	75	solid
P26	T3	YML	_config/concept_coverage.yaml [NEW]	85	ready
P27	T3	MD	_canon/README.md	85	ready
P28	T3	YML	_canon/counts.yaml	80	ready
P29	T3	YML	_canon/roster.yaml	80	ready
P30	T3	YML	_canon/taxonomy.yaml	75	solid
P31	T3	YML	_canon/timeline.yaml	75	solid
P32	T4	MD	_context/INGEST_PROCEDURE.md [NEW]	100	ready
P33	T4	MD	_context/SEMANTIC_LINT_PROCEDURE.md [NEW]	100	ready
P34	T4	MD	_context/CLAUDE_CONTEXT_RULES.md	80	ready
P35	T4	MD	Home.md	85	ready

P#	Tier	Type	Component	Score	Band
P36	T4	MD	00-Start-Here/Project-Overview.md	80	ready
P37	T4	MD	00-Start-Here/How-to-Use-This-Wiki.md	80	ready
P38	T4	MD	00-Start-Here/Glossary.md	80	ready
P39	T4	MD	00-Start-Here/Terminology-Rules.md	80	ready
P40	T4	MD	04-Contributing/Update-Cascade.md	80	ready
P41	T4	MD	04-Contributing/File-Routing.md	80	ready
P42	T4	MD	04-Contributing/Style-Guide.md	80	ready
P43	T4	MD	_brain_dump/README.md	90	ready
P44	T4	MD	_decisions/README.md	90	ready
P45	T4	MD	_decisions/ingest-log.md [NEW]	75	solid
P46	T4	MD	_inbox/README.md	90	ready
P47	T4	MD	_sources/README.md [v1.1]	85	ready
P48	T4	MD	_sources/raw/README.md [NEW]	85	ready
P49	T4	MD	_confidential/Confidential_Profile.md	70	solid
P50	T5	PY	bootstrap.py	70	solid
P51	T5	MD	README.md (Codex docs)	85	ready
P52	T5	OP	Bootstrap operation (one-time wiki creation)	75	solid
P53	T5	OP	Sync operation (cascade infra updates)	80	ready
P54	T5	OP	Ingest operation (source integration) [NEW]	75	solid

7.3 Score-vs-priority gaps (pause-and-clarify list)

Items where the readiness score is in the **solid** band (55–79) — meaning the design has open questions worth resolving with the user before writing code. Listed in build-order priority. None of these are blockers; all are candidates for a brief design conversation when their P# comes up.

P#	Component	Score	What to clarify
P10	check_cascade.py	75	mtime comparison source -> derived. Reads cascade_map.yaml.
P11	check_framework_briefing_sync.py	75	Pair-based mtime check (framework <-> public_pair).
P12	check_canon_consistency.py	70	Two modes: page-vs-canon (enforced) and page-vs-page (flagged).
P13	check_concept_coverage.py	75	Reads roster.yaml. Word-boundary case-sensitive matching with aliases. Threshold = pages-with-mentions, not total mentions.
P14	steel_thread_tracker.py	65	Multi-layer feature manifest. Schema for steel_threads.yaml needs care.
P15	build_canon_drift_report.py	65	Snapshot + diff over time. Storage location for snapshots = open question.
P16	delta_source_docs.py	60	Diff two source-doc versions + cascade impact. Most complex script.
P18	scaffold_source.py	70	Local file -> copy into _inbox/. URL -> placeholder + paste prompt (no fetch).
P20	sync_from_kit.py	70	v1.1 added _context/ procedure overwrite; CLAUDE_CONTEXT_RULES.md preserved.
P25	_config/steel_threads.yaml	75	Multi-layer feature manifest. Schema doc lives in spec section 2.5.
P30	_canon/taxonomy.yaml	75	Structured classifications. Format varies per project — keep skeleton flexible.
P31	_canon/timeline.yaml	75	Milestones + versions. Schema decisions left to per-project customization.
P45	_decisions/ingest-log.md	75	Append-only template; first heading + format example from INGEST_PROCEDURE section 6.
P49	_confidential/Confidential_Profile.md	70	Sensitive template. Eleven sections including Claude behavior rules.
P50	bootstrap.py	70	Scaffolder. Built after scripts + templates exist. Replaces ' ' with '/'. Idempotent.
P52	Bootstrap operation (one-time wiki creation)	75	Tested after bootstrap.py + all templates exist. v1.1 adds _sources/raw/ + procedures.
P54	Ingest operation (source integration)	75	Procedure file present; tested last because it depends on dashboards + canon.

8. Conflicts and Contradictions

Codex is built around the assumption that information will conflict — different sources will say different things, the wiki will diverge from canon, frameworks will drift from their public briefings. Codex doesn't try to prevent conflicts; it surfaces them and forces a human decision.

8.1 Sources of conflict Codex detects

Conflict type	Detected by	What happens
Page contradicts canon	check_canon_consistency.py	Page is flagged as inconsistent. Canon wins by definition.
Page contradicts another page (canon silent)	check_canon_consistency.py	Both pages flagged in a separate section. Resolution: promote the fact to <code>_canon/</code> , or rewrite one of the pages.
status: ready without canon_sources	validate_canon_integrity.py	Validator error. Page must cite a source or downgrade status.
unverified_claims non-empty on status: ready	validate_canon_integrity.py	Validator error. Resolve the claims or downgrade status.
Forbidden term in non-exempted page	validate_terminology.py	Validator error. Rewrite the page or set <code>allow_forbidden_terms: true</code> (escape hatch, used deliberately).
Public page leaks unreleased terminology	validate_reveal_conceit.py	Severity error/warning/info. info severity documents approved-public terms.
Source mtime newer than its derived doc	check_cascade.py	Cascade is stale. Run the update workflow before publishing.
Framework page newer than its public_pair	check_framework_briefing_sync.py	Briefing pair is stale. Update the public pair before publishing.
Broken <code>[[wikilink]]</code> or orphan page	check_cross_refs.py	Validator error or warning depending on severity.
Roster entity mentioned widely without dedicated page (v1.1)	check_concept_coverage.py	Reported in the concept-coverage dashboard. Suggests creating a page.
Ingest extraction conflicts with existing page	Ingest procedure step 3	Logged in the ingest log under contradictions. Never silently overwritten. User chooses: overwrite, quarantine to <code>_brain_dump/</code> , or no-op.
Ingest extraction conflicts with canon	Ingest procedure step 3	Refuses to run without explicit resolution. Same options as above.

8.2 Resolution strategies

- **Canon wins arbitrated conflicts.** If `_canon/` says the answer, pages must conform.
- **Sources win unarbitrated conflicts.** If two pages disagree and canon is silent, look at the cited sources to decide which wins, then promote the fact to canon to make the call durable.
- **Brain dump catches the unresolved.** When neither side of a contradiction is ready to commit, move the speculative claim to `_brain_dump/` with `dump_status: exploring`. It's quarantined, not lost.
- **Decisions log captures the choice.** When a non-trivial conflict is resolved, append an entry to `_decisions/` naming the options, the choice, and the rationale.

- **Escape hatches are deliberate.** `allow_forbidden_terms: true` exists for the page that needs to *discuss* a forbidden term (the terminology rules page itself, for example). Use it sparingly and document why in the page body.

8.3 What Codex never does

- Never silently overwrites canon during Ingest.
- Never edits archived sources in `_sources/raw/` (read-only after archive).
- Never promotes `_brain_dump/` entries to canon automatically — promotion is always explicit.
- Never modifies content folders during Sync (content stays put; only infrastructure flows).
- Never reverses the cascade direction (Source → Derived → Wiki, never the other way).

9. Project Files Reference

The four files provided in the build project, and how each is used:

File	Role	Disposition during build
CODEX_BUILD_SPEC_v1_1.md	Authoritative build specification	Read end-to-end before writing any code. Copied verbatim into <code>_codex/PROJECT_WIKI_BUILD_SPEC.md</code> (priority P3). Wins all contradictions.
codex-build-plan.html	Readiness scoring + build-order priority for all 54 components	Open before starting. Use the priority numbers (P1 → P54) as the build sequence. If a high-priority item has a low score, pause and resolve the design question before writing code.
INGEST_PROCEDURE.md	Claude-facing contract for the Ingest operation	Copied verbatim into <code>_template/_context/INGEST_PROCEDURE.md</code> (priority P32). Do not paraphrase, do not shorten. Bootstrap drops it into every new wiki; Sync overwrites it on every update.
SEMANTIC_LINT_PROCEDURE.md	Claude-facing contract for semantic lint runs	Copied verbatim into <code>_template/_context/SEMANTIC_LINT_PROCEDURE.md</code> (priority P33). Same disposition as <code>INGEST_PROCEDURE.md</code> .

9.1 The reference implementation question

The build spec describes Iron Soul / Planet Scoria Prime wiki v3.3 as a working reference implementation that can be used as a derivation source for scripts, configs, and most templates. **This Codex build is not derived from Iron Soul.** Iron Soul is the first *consumer* of Codex, not its source — so all scripts and templates must be built from the spec alone. The single exception remains the two shipped procedures (`INGEST_PROCEDURE.md` and `SEMANTIC_LINT_PROCEDURE.md`), which ship verbatim from the project files.

9.2 Cross-referencing during the build

When implementing a script, the order of authority is:

- Spec section 2.4 (script purpose, one-line contract)
- Spec section 5 (file manifest, exact filename and location)
- Spec section 7 Phase 3 (implementation details: stdlib only, code-block stripping, `WIKI_ROOT` path math, list-field parsing, etc.)
- Build plan note (anything specific to that component, especially v1.1 NEW items)
- Spec section 3 (design principles — never violate these)

When the build plan says "v1.1 NEW" on a component, that component has no precedent and must be implemented from spec text alone. Currently NEW items are: `check_concept_coverage.py`, `scaffold_source.py`, `concept_coverage.yaml`, the two shipped `_context/` procedures, `_decisions/ingest-log.md`, and `_sources/raw/README.md`.

10. Claude Code Behavior Rules

When the build spec is loaded as project knowledge and Codex is being built, these rules govern Claude Code's behavior.

10.1 Hard rules

- **Never use external Python packages.** Pure stdlib. Period.
- **Never edit files outside the Codex install path** unless the user explicitly directs (e.g. for self-test bootstraps to a temp path).
- **Always run a self-test bootstrap** before declaring Codex complete.
- **Always show the user the file manifest** at the end of the build.
- **Build templates from the spec.** Iron Soul is not present as a reference implementation in this project. The two shipped `_context/` procedure files are the only verbatim-included content.

10.2 Operating preferences

- Build in the order of section 6 (Phases 1–8). Don't write `bootstrap.py` before the scripts and templates exist.
- Test every script in isolation as soon as it's written, before moving to the next.
- Use the same frontmatter parser in every script (DRY — `_lib/frontmatter.py` is shared).
- Document operational caveats inline as code comments, especially around regex tips, escape hatches, and cross-platform path handling.

10.3 Red flags (stop and ask)

- The user asks for a feature not in the spec — confirm scope before adding.
- A script needs an external library — find a stdlib alternative or escalate.
- The reference implementation contradicts the spec — flag the conflict, ask which to follow.
- The two shipped procedures (`INGEST_PROCEDURE.md`, `SEMANTIC_LINT_PROCEDURE.md`) are missing from the build project — stop and request them; do not invent substitutes. (Currently both are present, so this is not a live concern.)

11. Procedures Reference

Both files in this section ship **verbatim** into every bootstrapped wiki's `_context/` folder. Do not paraphrase, do not shorten, do not 'improve'. The summaries below are reference only — the canonical text lives in the project files.

11.1 Ingest Procedure (summary)

Audience: Claude Code, when the user says 'ingest `_inbox/<source>`' or 'ingest the new source'.

Source formats: markdown, plaintext, PDF, or Web Clipper output. For non-markdown sources (PDFs especially), `scaffold_source.py` creates a sidecar `.md` in `_inbox/` next to the binary file.

Hard rules (seven):

- Never write to `_canon/*.yaml` without explicit confirmation.
- Never silently overwrite a page or canon entry. Flag contradictions; let the user choose.
- Never touch `_scripts/`, `_config/`, `_confidential/`, `04-Contributing/`, or unrelated content pages.
- Cite every new claim in `canon_sources` with a section reference where possible.
- Archive is irreversible. `_sources/raw/` is read-only after archive.
- One source per Ingest run. Multiple sources are ingested serially, each producing its own ingest-log entry.
- Versioned sources do not overwrite predecessors. `delta_source_docs.py` shows the cascade.

Procedure: 7 steps — Read → Extract → Route → Link → Archive → Log → Validate. Full text in the source file.

Refuses to run if: source frontmatter missing, uncommitted git changes, unresolved canon contradictions, or `_scripts/` has been edited since the last Sync.

11.2 Semantic Lint Procedure (summary)

Audience: Claude, when the user says 'lint the wiki' or 'run semantic lint'.

Authority: Complements the structural validators in `_scripts/`. Structural validators are enforcement (regex, frontmatter, link graph, canon integrity, cascade staleness). Semantic lint is inspection — it surfaces concerns regex cannot see.

Hard rules (six):

- Report only — never modify files. Output goes to `_dashboards/semantic_lint_report.md`.
- Read the wiki end to end before reporting. No partial reports.
- Cite every finding with page, section, and triggering evidence.
- Never flag `_brain_dump/` as contradicting canon (brain dump is explicitly not canon).
- Respect escape hatches like `allow_forbidden_terms: true`.
- Structural issues are out of scope (broken links, orphans, forbidden terms, canon integrity, cascade staleness, briefing-pair sync — those belong to validators).

Categories of findings:

- **A. Page-to-page contradictions** (unarbitrated) — promote-to-canon candidates.
- **B. Staleness** — page claims appear superseded by a newer source not yet reflected.
- **C. Concept gaps** — entities mentioned in 3+ pages with no dedicated page.

- **D. Canon gaps** — facts in 2+ pages that belong in `_canon/counts.yaml` or `_canon/roster.yaml`.
- **E. Tone / voice drift** — pages whose register is inconsistent with neighbors (conservative — only clear outliers).
- **F. Orphan reasoning** — decisions citing pages or facts that no longer exist.

When to run: monthly on active wikis; after any ingest batch larger than three sources; before considering any page status: ready; before sharing a public briefing pair externally; or when the user asks.

Appendix A: Frontmatter field reference

Quick reference for every YAML frontmatter field used in Codex wikis.

Field	Type	Notes
title	string	Required. Human-readable page title.
type	enum	framework reference guide overview episode dashboard profile brain_dump decision context_rules
visibility	enum	internal public confidential
completion	0-100	Drives the completion dashboard's red/yellow/green roll-up.
status	enum	gap (< 30) outlined (30–54) solid (55–79) ready (≥ 80). Pages with status: ready must have non-empty canon_sources and empty unverified_claims.
phase	0-N	Optional. Project-defined milestone marker.
last_updated	YYYY-MM-DD	Updated whenever the page changes substantively.
dependencies	list	Other pages this one logically depends on.
public_pair	string null	Path to the public briefing-guide pair, or null if none.
blocking_questions	list	Questions that must be answered before the page can advance to higher status.
canon_sources	list	Citations into _sources/raw/ (with optional section refs). REQUIRED for status: ready.
unverified_claims	list	Claims the page makes that haven't been backed by a source yet. MUST be empty for status: ready.
source	string	Free-text citation (legacy field, still supported).
allow_forbidden_terms	bool	Optional escape hatch. Set true to exempt a page from terminology validation (use sparingly; document the reason in the body).
generated	bool	Auto-generated dashboards only. Validators and consumers may treat these as ephemeral.
dump_status	enum	Brain dump only. exploring validated migrated superseded rejected.
migrated_to	list	Brain dump only. Paths the entry was promoted into when validated/migrated.
source	string	_inbox/ and _sources/raw/ frontmatter. The original source citation.
ingested_date	YYYY-MM-DD	_inbox/ and _sources/raw/ frontmatter. Date the source was scaffolded.
status	enum	_inbox/ and _sources/raw/ frontmatter. pending_triage ingested. Distinct from the content-page status enum.

Appendix B: Quick command reference

After Codex v1.0 is in production, consuming projects use these commands.

Bootstrap a new wiki

```
cd <Codex install path>
python bootstrap.py <new-wiki-path>

# Customize the new wiki:
# 1. Rename 01-Domain-1/, 02-Domain-2/ to project domains
# 2. Populate _config/*.yaml with project rules
# 3. Populate _canon/*.yaml with project facts
# 4. Customize _confidential/Confidential_Profile.md
# 5. Customize _context/CLAUDE_CONTEXT_RULES.md
# 6. Customize Home.md and 00-Start-Here/ pages
```

Run dashboards (after any change)

```
cd <new-wiki-path>
python _scripts/update_dashboards.py
```

Ingest a new source (v1.1, ongoing)

```
python _scripts/scaffold_source.py <path-to-source-or-URL>
# Then in Claude Code: "Ingest _inbox/<source>.md"
```

Semantic lint (v1.1, periodic)

```
# In Claude Code: "Run semantic lint"
# Writes to _dashboards/semantic_lint_report.md
```

Sync with updated Codex (later, when Codex improves)

```
cd <wiki-path>
python _scripts/sync_from_kit.py <Codex install path> --dry-run
python _scripts/sync_from_kit.py <Codex install path>
python _scripts/update_dashboards.py
```

End of document.